

2. Matriz de escenarios

Cada fila es una instrucción; la columna **n** es el valor de la variable compartida **después** de ejecutarla. En negrita, la guarda que resulta falsa y termina al proceso.

2.1. 8(a) · el bucle de p se ejecuta exactamente una vez

#	p	q	n
1		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
2		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
3	p1 while $n < 1 - 0 < 1 \rightarrow$ verdadero		0
4	p2 $n \leftarrow n + 1 - n = 0 + 1$		1
5	p1 while $n < 1 - 1 < 1 \rightarrow$ falso		1
6		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
7		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
8		q1 while $n \geq 0 - 0 \geq 0 \rightarrow$ verdadero	0
9		q2 $n \leftarrow n - 1 - n = 0 - 1$	-1
10		q1 while $n \geq 0 - -1 \geq 0 \rightarrow$ falso	-1

q entra a su bucle y deja $n = 0$, así que la guarda de p ahora es verdadera. p incrementa una vez y n vuelve a 1, con lo que su guarda se vuelve falsa y termina. q sigue solo hasta $n = -1$.

2.2. 8(b) · el bucle de p se ejecuta exactamente tres veces

#	p	q	n
1		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
2		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
3	p1 while $n < 1 - 0 < 1 \rightarrow$ verdadero		0
4	p2 $n \leftarrow n + 1 - n = 0 + 1$		1
5		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
6		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
7	p1 while $n < 1 - 0 < 1 \rightarrow$ verdadero		0
8	p2 $n \leftarrow n + 1 - n = 0 + 1$		1
9		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
10		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
11	p1 while $n < 1 - 0 < 1 \rightarrow$ verdadero		0
12	p2 $n \leftarrow n + 1 - n = 0 + 1$		1
13	p1 while $n < 1 - 1 < 1 \rightarrow$ falso		1
14		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
15		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
16		q1 while $n \geq 0 - 0 \geq 0 \rightarrow$ verdadero	0
17		q2 $n \leftarrow n - 1 - n = 0 - 1$	-1
18		q1 while $n \geq 0 - -1 \geq 0 \rightarrow$ falso	-1

El mismo patrón tres veces: cada vez que q baja n a 0, p lo sube a 1. En la cuarta evaluación p encuentra $n = 1$ porque q no volvió a decrementar antes, y termina. El número de iteraciones de p no es una propiedad del programa: lo elige el entrelazado.

2.3. 8(c) · los dos bucles se ejecutan infinitas veces

#	p	q	n
1		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
2		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
3	p1 while $n < 1 - 0 < 1 \rightarrow$ verdadero		0
4	p2 $n \leftarrow n + 1 - n = 0 + 1$		1
5		q1 while $n \geq 0 - 1 \geq 0 \rightarrow$ verdadero	1
6		q2 $n \leftarrow n - 1 - n = 1 - 1$	0
7	p1 while $n < 1 - 0 < 1 \rightarrow$ verdadero		0

#	p	q	n
8	p2 n <- n + 1 — n = 0 + 1		1
9		q1 while n >= 0 — 1 >= 0 → verdadero	1
10		q2 n <- n - 1 — n = 1 - 1	0
11	p1 while n < 1 — 0 < 1 → verdadero		0
12	p2 n <- n + 1 — n = 0 + 1		1

Cada vuelta qppp (bandas alternadas) deja $n = 1$ y a los dos procesos otra vez en su guarda (p1, q1): el estado es idéntico al inicial y en la vuelta iteraron los dos bucles, sin que ninguno termine. Por inducción, repetir la vuelta siempre es posible, así que el escenario infinito existe. Ninguna guarda se viola nunca (*safety*), y sin embargo el programa puede no terminar (*liveness*). La verificación sobre **todos** los entrelazados, no solo estos tres, corresponde a Spin.

3. La carrera real con goroutines

El modelo anterior elige el entrelazado a mano. week04/cmd/race ejecuta el mismo algoritmo con dos goroutines sobre n sin sincronización; el detector de carreras de Go la reporta y el número de iteraciones cambia entre corridas. Salida de `go run -race ./week04/cmd/race` (recortada):

```
=====
WARNING: DATA RACE
Read at 0x00c00011e020 by goroutine 7:
    main.main.func1()
        /home/roswt-dell/Code/cursos/concurrente/labs/go/week04/cmd/race/main.go:40 +0x99

Previous write at 0x00c00011e020 by goroutine 8:
    main.main.func2()
        /home/roswt-dell/Code/cursos/concurrente/labs/go/week04/cmd/race/main.go:51 +0xbb

Goroutine 7 (running) created at:
    main.main()
        /home/roswt-dell/Code/cursos/concurrente/labs/go/week04/cmd/race/main.go:38 +0x1c4

Goroutine 8 (finished) created at:
    main.main()
        /home/roswt-dell/Code/cursos/concurrente/labs/go/week04/cmd/race/main.go:48 +0x2ae
=====
n final = 1
iteraciones: p = 2, q = 2
compará con week04/ESCENARIOS.md, donde el entrelazado se elige a mano
Found 1 data race(s)
exit status 66
```

4. Verificación

```
cd labs/go
go test -race ./week04/...      # 12 tests: escenarios 8(a-c), traza, tablas y CSV
go run ./week04/cmd/escenarios  # regenera ESCENARIOS.md (un test falla si queda viejo)
go run -race ./week04/cmd/race  # la carrera de datos real
```

5. Anexo: código Go

5.1. week04/escenarios.go — modelo determinista

```
// Package week04 resuelve el ejercicio 8 del capítulo 2 de Ben-Ari sobre el
// algoritmo 2.18 (algoritmo concurrente C):
//
// integer n <- 1
// p                q
// p1: while n < 1    q1: while n >= 0
// p2:    n <- n + 1    q2:    n <- n - 1
//
// Un escenario, en el sentido de Ben-Ari, es un entrelazado concreto: quién
// ejecuta cada instrucción y qué valor toma n. Acá se construyen a mano y se
// ejecutan de forma determinista, porque el ejercicio pide *exhibir* un
// entrelazado, no observar uno al azar.
//
// Por eso el modelo no usa goroutines: con goroutines el planificador elige y
// no se puede pedir "el bucle de p exactamente tres veces". La versión con
// goroutines y su carrera de datos está en cmd/race, y la verificación sobre
// *todos* los entrelazados, en ../../spin/.
package week04

import (
    "encoding/csv"
    "fmt"
    "strconv"
    "strings"
)

// Escenarios del ejercicio 8. Cada carácter es un turno: qué proceso ejecuta
// su siguiente instrucción.
const (
    // EscenarioA: el bucle de p se ejecuta exactamente una vez (8a).
    EscenarioA = "qqppppqqqqq"
    // EscenarioB: el bucle de p se ejecuta exactamente tres veces (8b).
    EscenarioB = "qqppqqppppppppqqqqq"
    // EscenarioC: una vuelta que devuelve el estado al inicial habiendo
    // iterado los dos bucles. Repetida para siempre, ninguno termina (8c).
    EscenarioC = "qqpp"
)

// Posiciones del contador de programa de cada proceso.
const (
    enGuarda = iota // p1 / q1
    enCuerpo         // p2 / q2
    terminado
)

// Paso es una fila de la tabla de escenarios: la instrucción que se ejecutó y
// el valor de n inmediatamente después.
type Paso struct {
    Proceso string // "p" o "q"
    Etiqueta string // "p1", "p2", "q1", "q2"
   Codigo  string // la instrucción tal como aparece en el algoritmo
   Detalle string // la guarda evaluada o la asignación hecha, con valores
    N      int
}

// Traza es la sucesión de pasos de un escenario.
type Traza []Paso

// Estado es lo que hace falta para saber si un escenario volvió al inicio.
type Estado struct {
    N      int
    PC     map[string]int
    iteras map[string]int
}
```

```

// Ejecucion mantiene el estado compartido y el contador de programa de cada
// proceso mientras se recorre un escenario.
type Ejecucion struct {
    n      int
    pc     map[string]int
    iteras map[string]int
}

// NuevaEjecucion arranca con n = 1 y los dos procesos en su guarda.
func NuevaEjecucion() *Ejecucion {
    return &Ejecucion{
        n:      1,
        pc:     map[string]int{"p": enGuarda, "q": enGuarda},
        iteras: map[string]int{"p": 0, "q": 0},
    }
}

// N es el valor actual de la variable compartida.
func (e *Ejecucion) N() int { return e.n }

// Iteraciones cuenta cuántas veces ejecutó su cuerpo el bucle del proceso.
func (e *Ejecucion) Iteraciones(proceso string) int { return e.iteras[proceso] }

// Termino indica si la guarda del proceso ya resultó falsa.
func (e *Ejecucion) Termino(proceso string) bool { return e.pc[proceso] == terminado }

// Paso ejecuta la siguiente instrucción del proceso y devuelve la fila
// correspondiente de la tabla.
func (e *Ejecucion) Paso(proceso string) (Paso, error) {
    pc, conocido := e.pc[proceso]
    if !conocido {
        return Paso{}, fmt.Errorf("proceso desconocido %q: solo hay p y q", proceso)
    }

    switch pc {
    case terminado:
        return Paso{}, fmt.Errorf("el proceso %s ya terminó: su guarda resultó falsa", proceso)

    case enGuarda:
        anterior := e.n
        vale := e.guarda(proceso)
        if vale {
            e.pc[proceso] = enCuerpo
        } else {
            e.pc[proceso] = terminado
        }
        return Paso{
            Proceso: proceso,
            Etiqueta: proceso + "1",
            Codigo:   e.codigoGuarda(proceso),
            Detalle:  fmt.Sprintf("%s → %s", e.guardaConValores(proceso, anterior), veredicto(vale)),
            N:        e.n,
        }, nil

    default: // enCuerpo
        anterior := e.n
        if proceso == "p" {
            e.n++
        } else {
            e.n--
        }
        e.iteras[proceso]++
        e.pc[proceso] = enGuarda
        return Paso{
            Proceso: proceso,
            Etiqueta: proceso + "2",
        }, nil
    }
}

```

```

        Codigo: e.codigoCuerpo(proceso),
        Detalle: fmt.Sprintf("n = %d %s 1", anterior, signo(proceso)),
        N: e.n,
    }, nil
}
}

// Correr ejecuta un escenario completo, un turno por carácter.
func (e *Ejecucion) Correr(escenario string) (Traza, error) {
    traza := make(Traza, 0, len(escenario))
    for i, turno := range escenario {
        paso, err := e.Paso(string(turno))
        if err != nil {
            return traza, fmt.Errorf("turno %d (%q): %w", i+1, string(turno), err)
        }
        traza = append(traza, paso)
    }
    return traza, nil
}

// Estado devuelve una copia del estado, para comparar el inicio con el final.
func (e *Ejecucion) Estado() Estado {
    return Estado{
        N: e.n,
        PC: map[string]int{"p": e.pc["p"], "q": e.pc["q"]},
        iteras: map[string]int{"p": e.iteras["p"], "q": e.iteras["q"]},
    }
}

// Ciclo resume una vuelta candidata a repetirse para siempre.
type Ciclo struct {
    Traza Traza
    VuelveAlInicio bool // n y los contadores de programa quedaron como al inicio
    IteracionesP int
    IteracionesQ int
    TerminoAlguno bool
}

// AnalizarCiclo corre el escenario una vez y reporta si puede repetirse para
// siempre: basta que deje el estado igual al inicial, que ningún proceso haya
// terminado y que los dos bucles hayan iterado. Eso responde 8(c) sin
// simular infinitos pasos.
func AnalizarCiclo(escenario string) (Ciclo, error) {
    e := NuevaEjecucion()
    inicial := e.Estado()

    traza, err := e.Correr(escenario)
    if err != nil {
        return Ciclo{}, err
    }
    final := e.Estado()

    return Ciclo{
        Traza: traza,
        VuelveAlInicio: final.N == inicial.N && final.PC["p"] == inicial.PC["p"] && final.PC["q"] ==
inicial.PC["q"],
        IteracionesP: e.Iteraciones("p"),
        IteracionesQ: e.Iteraciones("q"),
        TerminoAlguno: e.Termino("p") || e.Termino("q"),
    }, nil
}

// Markdown arma la matriz de escenarios: una columna por proceso, como en las
// tablas de Ben-Ari, y el valor de n después de cada instrucción.
func Markdown(t Traza) string {
    var b strings.Builder
    b.WriteString("| # | p | q | n |\n")

```

```

b.WriteString("|---|---|---|---|\n")
for i, paso := range t {
    celda := fmt.Sprintf("`%s` %s - %s", paso.Etiqueta, paso.Codigo, paso.Detalle)
    p, q := "", ""
    if paso.Proceso == "p" {
        p = celda
    } else {
        q = celda
    }
    fmt.Fprintf(&b, "| %d | %s | %s | %d |\n", i+1, p, q, paso.N)
}
return b.String()
}

// CSV exporta la traza con encabezado, una fila por paso. Es la entrada del
// gráfico de la entrega (grafico/escenarios.py).
func CSV(escenario string, t Traza) string {
    return "escenario,paso,proceso,etiqueta,detalle,n\n" + FilasCSV(escenario, t)
}

// FilasCSV es CSV sin el encabezado, para juntar varios escenarios en un
// mismo archivo.
func FilasCSV(escenario string, t Traza) string {
    var b strings.Builder
    w := csv.NewWriter(&b)
    for i, paso := range t {
        // Escribir en un strings.Builder no falla; el error de csv se revisa abajo.
        _ = w.Write([]string{
            escenario, strconv.Itoa(i + 1), paso.Proceso, paso.Etiqueta,
            paso.Detalle, strconv.Itoa(paso.N),
        })
    }
    w.Flush()
    if err := w.Error(); err != nil {
        panic(err)
    }
    return b.String()
}

func (e *Ejecucion) guarda(proceso string) bool {
    if proceso == "p" {
        return e.n < 1
    }
    return e.n >= 0
}

func (e *Ejecucion) codigoGuarda(proceso string) string {
    if proceso == "p" {
        return "while n < 1"
    }
    return "while n >= 0"
}

func (e *Ejecucion) codigoCuerpo(proceso string) string {
    if proceso == "p" {
        return "n <- n + 1"
    }
    return "n <- n - 1"
}

func (e *Ejecucion) guardaConValores(proceso string, n int) string {
    if proceso == "p" {
        return fmt.Sprintf("%d < 1", n)
    }
    return fmt.Sprintf("%d >= 0", n)
}

```

```
func signo(proceso string) string {
    if proceso == "p" {
        return "+"
    }
    return "-"
}

func veredicto(vale bool) string {
    if vale {
        return "verdadero"
    }
    return "falso"
}
```

5.2. week04/escenarios_test.go — tests

```
package week04

import (
    "os"
    "strings"
    "testing"
)

// Los escenarios se escriben como una cadena de turnos: cada carácter dice qué
// proceso ejecuta su siguiente instrucción. "qqppp" = q, q, p, p, p.

func TestSinInterferenciaPNuncaEntraAlBucle(t *testing.T) {
    // n arranca en 1, así que la guarda n < 1 de p es falsa desde el principio.
    e := NuevaEjecucion()
    if _, err := e.Correr("p"); err != nil {
        t.Fatalf("Correr: %v", err)
    }

    if !e.Termino("p") {
        t.Error("p debería haber terminado con la primera evaluación de p1")
    }
    if got := e.Iteraciones("p"); got != 0 {
        t.Errorf("iteraciones de p = %d, se esperaba 0", got)
    }
    if got := e.N(); got != 1 {
        t.Errorf("n = %d, se esperaba 1 (p no alcanzó a incrementar)", got)
    }
}

func TestSinInterferenciaQIteraDosVeces(t *testing.T) {
    // q decrementa mientras n >= 0: 1 → 0 → -1 y la guarda se vuelve falsa.
    e := NuevaEjecucion()
    if _, err := e.Correr("qqqqq"); err != nil {
        t.Fatalf("Correr: %v", err)
    }

    if !e.Termino("q") {
        t.Error("q debería haber terminado")
    }
    if got := e.Iteraciones("q"); got != 2 {
        t.Errorf("iteraciones de q = %d, se esperaba 2", got)
    }
    if got := e.N(); got != -1 {
        t.Errorf("n = %d, se esperaba -1", got)
    }
}

// Ejercicio 8(a): un escenario donde el bucle de p se ejecuta exactamente una vez.
func TestEscenarioA_ElBucleDePSeEjecutaUnaVez(t *testing.T) {
    e := NuevaEjecucion()
}
```

```

traza, err := e.Correr(EscenarioA)
if err != nil {
    t.Fatalf("Correr: %v", err)
}

if got := e.Iteraciones("p"); got != 1 {
    t.Errorf("iteraciones de p = %d, se esperaba 1", got)
}
if !e.Termino("p") || !e.Termino("q") {
    t.Errorf("los dos procesos deberían terminar: p=%v q=%v", e.Termino("p"), e.Termino("q"))
}
if got := len(traza); got != len(EscenarioA) {
    t.Errorf("la traza tiene %d pasos y el escenario %d", got, len(EscenarioA))
}
}

// Ejercicio 8(b): un escenario donde el bucle de p se ejecuta exactamente tres veces.
func TestEscenarioB_ElBucleDePSeEjecutaTresVeces(t *testing.T) {
    e := NuevaEjecucion()
    if _, err := e.Correr(EscenarioB); err != nil {
        t.Fatalf("Correr: %v", err)
    }

    if got := e.Iteraciones("p"); got != 3 {
        t.Errorf("iteraciones de p = %d, se esperaba 3", got)
    }
    if !e.Termino("p") || !e.Termino("q") {
        t.Errorf("los dos procesos deberían terminar: p=%v q=%v", e.Termino("p"), e.Termino("q"))
    }
}

// Ejercicio 8(c): un escenario donde los dos bucles se ejecutan infinitas veces.
// No se puede "correr" un escenario infinito: lo que se demuestra es que el
// ciclo devuelve el estado al inicial habiendo iterado los dos bucles, así que
// repetirlo para siempre es un escenario válido.
func TestEscenarioC_AmbosBuclesIteranInfinitamente(t *testing.T) {
    ciclo, err := AnalizarCiclo(EscenarioC)
    if err != nil {
        t.Fatalf("AnalizarCiclo: %v", err)
    }

    if !ciclo.VuelveAlInicio {
        t.Error("el escenario debería dejar n y los contadores de programa como al inicio")
    }
    if ciclo.IteracionesP < 1 || ciclo.IteracionesQ < 1 {
        t.Errorf("cada vuelta debe iterar los dos bucles: p=%d q=%d",
            ciclo.IteracionesP, ciclo.IteracionesQ)
    }
    if ciclo.TerminoAlguno {
        t.Error("ningún proceso puede terminar dentro del ciclo")
    }
}

func TestNoSePuedeAvanzarUnProcesoTerminado(t *testing.T) {
    e := NuevaEjecucion()
    if _, err := e.Correr("p"); err != nil { // p termina en su primera guarda
        t.Fatalf("Correr: %v", err)
    }

    if _, err := e.Paso("p"); err == nil {
        t.Error("avanzar p después de que terminó debería dar error")
    }
}

func TestTurnoDesconocidoDaError(t *testing.T) {
    e := NuevaEjecucion()
    if _, err := e.Correr("x"); err == nil {

```

```

    t.Error("un turno que no es p ni q debería dar error")
}
}

func TestLaTrazaRegistraGuardasYAsignaciones(t *testing.T) {
    e := NuevaEjecucion()
    traza, err := e.Correr("qqp")
    if err != nil {
        t.Fatalf("Correr: %v", err)
    }

    // Valores calculados a mano: q1 con n=1 es verdadera, q2 deja n=0,
    // y entonces la guarda de p (n < 1) pasa a ser verdadera.
    quiero := []Paso{
        {Proceso: "q", Etiqueta: "q1",Codigo: "while n >= 0", Detalle: "1 >= 0 → verdadero", N: 1},
        {Proceso: "q", Etiqueta: "q2",Codigo: "n <- n - 1", Detalle: "n = 1 - 1", N: 0},
        {Proceso: "p", Etiqueta: "p1",Codigo: "while n < 1", Detalle: "0 < 1 → verdadero", N: 0},
    }
    if len(traza) != len(quiero) {
        t.Fatalf("la traza tiene %d pasos, se esperaban %d", len(traza), len(quiero))
    }
    for i, p := range quiero {
        if traza[i] != p {
            t.Errorf("paso %d = %v, se esperaba %v", i+1, traza[i], p)
        }
    }
}

func TestMarkdownEsLaTablaDeEscenarios(t *testing.T) {
    e := NuevaEjecucion()
    traza, err := e.Correr("qp")
    if err != nil {
        t.Fatalf("Correr: %v", err)
    }

    got := Markdown(traza)
    quiero := strings.Join([]string{
        "| # | p | q | n |",
        "|--|--|--|--|",
        "| 1 | | `q1` while n >= 0 - 1 >= 0 → verdadero | 1 |",
        "| 2 | `p1` while n < 1 - 1 < 1 → falso | | 1 |",
        "",
    }, "\n")
    if got != quiero {
        t.Errorf("Markdown devolvió:\n%s\nse esperaba:\n%s", got, quiero)
    }
}

// El gráfico de la entrega se dibuja desde este CSV, así que sale del mismo
// modelo que las tablas y no puede contradecirlas.
func TestCSVTraeUnaFilaPorPaso(t *testing.T) {
    e := NuevaEjecucion()
    traza, err := e.Correr("qp")
    if err != nil {
        t.Fatalf("Correr: %v", err)
    }

    got := CSV("8x", traza)
    quiero := strings.Join([]string{
        "escenario,paso,proceso,etiqueta,detalle,n",
        "8x,1,q,q1,1 >= 0 → verdadero,1",
        "8x,2,p,p1,1 < 1 → falso,1",
        "",
    }, "\n")
    if got != quiero {
        t.Errorf("CSV devolvió:\n%s\nse esperaba:\n%s", got, quiero)
    }
}

```

```

}

func TestCSVSinEncabezadoParaConcatenar(t *testing.T) {
    e := NuevaEjecucion()
    traza, err := e.Correr("q")
    if err != nil {
        t.Fatalf("Correr: %v", err)
    }

    if got := FilasCSV("8x", traza); got != "8x,1,q,q1,1 >= 0 → verdadero,1\n" {
        t.Errorf("FilasCSV = %q", got)
    }
}

// La matriz de escenarios que se entrega es un archivo generado: si el modelo
// cambia, este test falla en vez de dejar el .md desactualizado.
func TestArchivoEscenariosEstaAlDia(t *testing.T) {
    enDisco, err := os.ReadFile("ESCENARIOS.md")
    if err != nil {
        t.Fatalf("Leer ESCENARIOS.md: %v", err)
    }

    if string(enDisco) != Documento() {
        t.Error("ESCENARIOS.md está desactualizado: regeneralo con " +
            "`go run ./week04/cmd/escenarios > week04/ESCENARIOS.md`")
    }
}

func TestDocumentoIncluyeLosTresEscenarios(t *testing.T) {
    doc := Documento()
    for _, titulo := range []string{"8(a)", "8(b)", "8(c)"} {
        if !strings.Contains(doc, titulo) {
            t.Errorf("el documento no menciona el escenario %s", titulo)
        }
    }
    if !strings.Contains(doc, "| # | p | q | n |") {
        t.Error("el documento debería traer la tabla de escenarios")
    }
}

```

5.3. week04/cmd/race/main.go — versión con goroutines

```

// Command race ejecuta el algoritmo concurrente C con goroutines de verdad,
// para mostrar lo que el modelo determinista de week04 no puede mostrar: que
// sobre memoria compartida sin sincronización esto es una carrera de datos.
//
// go run -race ./week04/cmd/race
//
// Qué esperar:
//
// - Con -race: un reporte "WARNING: DATA RACE" sobre n, con la lectura de una
//   guarda y la escritura de la otra goroutine.
// - El resultado cambia entre corridas: a veces p no itera nunca, a veces
//   itera muchas veces. Ese número no es una propiedad del programa, lo elige
//   el planificador – que es justo lo que el ejercicio 8 pide construir a
//   mano en ESCENARIOS.md.
// - Puede chocar con el tope de pasos: el escenario infinito de 8(c) no es
//   teórico, y sin el tope esto no terminaría.
//
// Ojo con una trampa extra: nada obliga al compilador a releer n en cada vuelta
// del bucle. Sin sincronización, una goroutine puede quedarse con un valor
// cacheado y girar para siempre aunque la otra ya haya cambiado la variable.
package main

import (
    "fmt"

```

```

    "sync"
)

// tope corta la corrida: el algoritmo puede no terminar (ejercicio 8c).
const tope = 10_000_000

func main() {
    n := 1 // la variable compartida del algoritmo
    var iterP, iterQ int

    var wg sync.WaitGroup
    wg.Add(2)

    go func() { // p1: while n < 1 / p2: n <- n + 1
        defer wg.Done()
        for n < 1 {
            n++
            if iterP++; iterP >= tope {
                return
            }
        }
    }()

    go func() { // q1: while n >= 0 / q2: n <- n - 1
        defer wg.Done()
        for n >= 0 {
            n--
            if iterQ++; iterQ >= tope {
                return
            }
        }
    }()

    wg.Wait()

    fmt.Printf("n final = %d\n", n)
    fmt.Printf("iteraciones: p = %d, q = %d\n", iterP, iterQ)
    if iterP >= tope || iterQ >= tope {
        fmt.Printf("se alcanzó el tope de %d pasos: este entrelazado no terminaba\n", tope)
    }
    fmt.Println("compará con week04/ESCENARIOS.md, donde el entrelazado se elige a mano")
}

```